Introduction to Scientific Programming

Lecture 3: *Development Tools & Workflows*

Renato Duarte

@Renatocfduarte
rcfduarte.github.io
@rcfduarte
renato.duarte@cnc.uc.pt

31/10/2025

CENTER FOR NEUROSCIENCE AND CELL BIOLOGY
UNIVERSITY OF COIMBRA
PORTUGAL

1 2 9 0 UNIVERSIDADE D COIMBRA

Day 3: Development Tools & Workflows

Lecture:

1. Code editors and IDEs
2. Notebooks & interactive programming
3. Documentation with Markdown and docstrings
4. Virtual environments (conda, poetry, uv)
5. Git fundamentals and GitHub workflows
6. Summary: Computational Research Workflows

Practical:

- Handout & Tutorial exercises

Code Editors & IDEs

What are they?

Code Editor:

A **text editor** with features specifically designed for writing code:

- Syntax highlighting
- Auto-completion
- Basic file management
- Lightweight and fast
- Extensible through plugins

A specialized notepad for programmers

IDE (Integrated Development Environment)

A **complete software suite** that combines multiple development tools:

- Code editor + debugger + compiler
- Project management
- Built-in terminal
- Testing frameworks
- Version control integration

A complete workshop with all tools included

Note: *Modern code editors (like VS Code) blur the line by offering IDE-like features through extensions.*

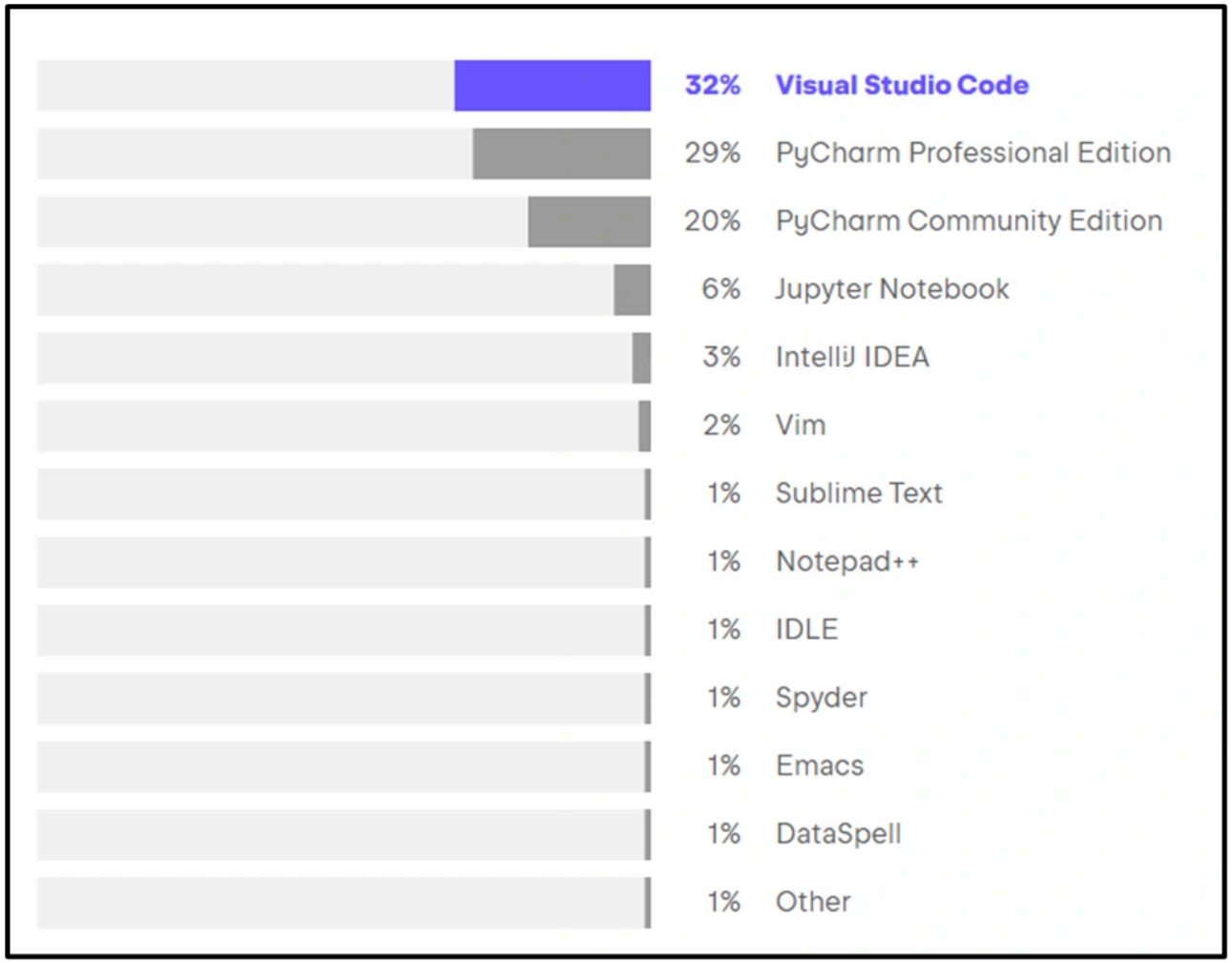
Code Editors & IDEs

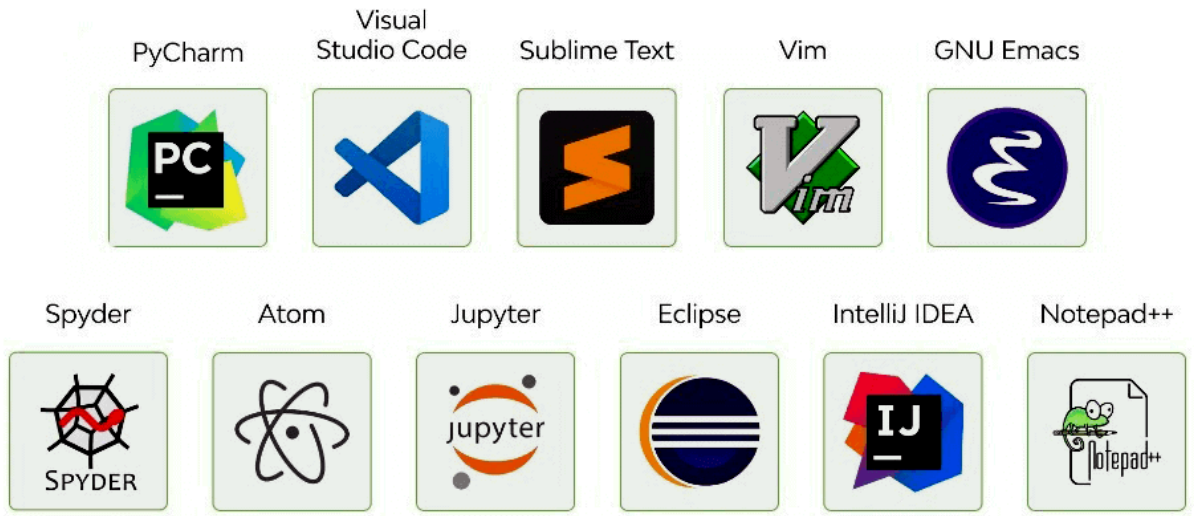
What are they?

	Code editors	IDEs
Feature set	Provides standard code editing functionality, with modern code editors including more advanced functionality.	Provides comprehensive functionality.
Language support	Polyglot with support for additional languages through plugins or extensions.	Provides extensive support for specific programming languages and frameworks.
Extensibility	Lightweight feature set that can be built on with community-driven extensions.	Large feature set already built in with APIs for developing community extensions.
Resource usage	Consumes fewer system resources.	Consumes more system resources.
Learning curve	Generally simpler, less complex, and easier to learn.	Generally a wider range of features, more complex, and harder to learn.
Integration with external tools	Integrates with external tools through plugins or extensions, which can need some configuration.	Preconfigured integration with external tools and services commonly used in software development.
Examples	VS Code, Cursor, Zed, Nova, Sublime Text**, Vim**, Notepad++** **Text editors	JetBrains IDEs (IntelliJ IDEA, WebStorm, PhpStorm, PyCharm, Rider), Visual Studio, Eclipse

Code Editors & IDEs

Which one to choose?









Code Editors & IDEs

Which one to choose?



✓ Strengths

- **Lightweight:** Fast startup, low memory
- **Extensions:** 50k+ available
- **Git integration:** built-in version control
- **IntelliSense:** Smart code completion
- **Debugger:** Powerful debugger included
- **Terminal:** Integrated console
- **Cross-platform:** Windows, Max, Linux
- **Community:** Huge user base, support
- **Integrated AI:** GitHub Co-Pilot, most dedicated AI IDEs are forks of VS code

⚠ Considerations

- **Setup required:** Need to configure extensions for Python
- **Extension overload:** Can be overwhelming for beginners
- **Memory with extensions:** Heavy with many plugins
- **Not Python-specific:** Generic, not designed only for Python

Recommended Extensions:

Python, Pylance, Jupyter, Python Debugger

Verdict

Excellent choice for beginners and professionals. Most versatile option with best extension ecosystem. Recommended for this course.

Code Editors & IDEs

Which one to choose?



PyCharm

JETBRAINS IDE

✓ Strengths

- **Python-focused:** Built specifically for Python
- **Smart code assistance:** Best-in-class IntelliSense
- **Debugging:** Most powerful Python debugger
- **Testing:** Integrated test runners (pytest, unittest)
- **Refactoring:** Advanced code restructuring
- **Scientific tools:** Data viewer, plots support
- **Database tools:** SQL support (Pro only)
- **Ready to use:** No configuration needed

⚠ Considerations

- **Heavy resource usage:** Requires 8GB+ RAM
- **Slow startup:** Can take 10-30 seconds
- **Steep learning curve:** Many features to learn
- **Professional cost:** €249/year (Free for education)
- **Python-only:** Not ideal for multi-language projects

Educational License:

Free Professional edition for students/teachers with edu email

Verdict

Best professional Python IDE. Excellent if you have resources and focus only on Python. Free Community edition sufficient for beginners.

Code Editors & IDEs

Which one to choose?

spyder

"Scientific Python Development Environment"

IDE designed for scientists and engineers

Best for: Scientists transitioning from MATLAB

⚠ **Considerations:** Slower, less popular or maintained



Website: spider-ide.org
License: Free (MIT)

Code Editors & IDEs

Which one to choose?



Sophisticated text editor with Python support;
Fastest and elegant

Best for: Quick edits, lightweight coding

⚠ **Considerations:** Fewer Python-specific features, smaller community

Website: sublimetext.com
License: Closed (paid with free version)

Code Editors & IDEs

Which one to choose?

🎓 Beginner / Learner

Recommended: VS Code

- Easy to learn, lightweight
- Great documentation & tutorials
- Adaptable & customizable

Alternative: PyCharm Community

📊 Data Science / Research

Recommended: VS Code + Jupyter

- Jupyter for exploratory data analysis
- VS Code for scripting and module development

Alternative: Spyder

💼 Professional Development

Recommended: PyCharm

- Best Python tooling available
- Advanced debugging & refactoring
- Professional features (DB, web, ...)

Alternative: VS Code

Primary tool: VS Code

For all generic programming

Secondary tool: Jupyter

For data exploration

The best IDE is the one you're comfortable with!

Notebooks & interactive programming

What is it?

Traditional Approach

1. Write entire script
2. Save file (.py)
3. Run complete program
4. Wait for all results
5. Find errors
6. Edit and repeat

⚠️ Re-run everything each time

Interactive Approach

1. Write small code snippet
2. Execute immediately
3. See result instantly
4. Adjust based on output
5. Continue building
6. Iterate incrementally

✓ Variables stay in memory

Why it matters?

- Rapid hypothesis testing, immediate visualization, and literate workflows
- Seemless integration of code, data and explanatory text

Notebooks & interactive programming

What is it?



Interactive documents that combine **executable code**, **rich output** (plots, tables, images), and **explanatory text** in a single document.

Code Cells	Output Cells	Markdown Cells
Executable Python code	Results, plots, tables	Documentation, notes

Why it matters?

- Rapid hypothesis testing, immediate visualization, and literate workflows
- Seemless integration of code, data and explanatory text

Notebooks & interactive programming

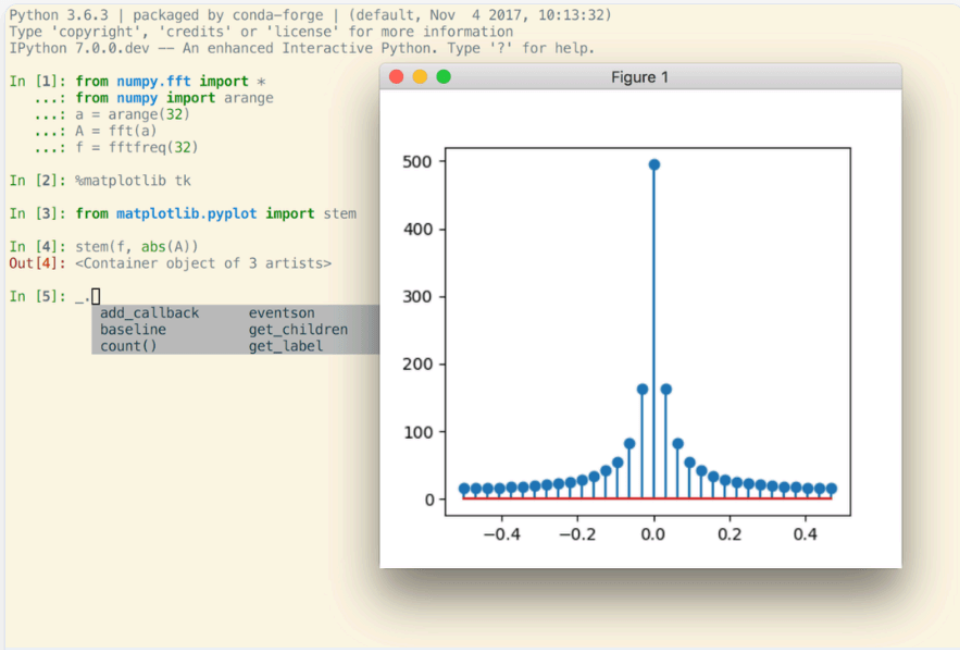
Brief history: REPL→Jupyter

- **1960s–1970s:**
  Interactive terminals and REPL (Read–Eval–Print Loop) emerge
- **1980s–2000s:**
  Mathematica, Maple, SageMath popularize computational notebooks
- **2001–2013:**
  IPython evolves as rich interactive Python computing environment
- **2011:**
  IPython Notebook introduces browser-based computational interface
- **2014:**
  Project Jupyter spins off from IPython, becomes language-agnostic
- **2018→:**
  JupyterLab released as next-gen UI with multi-language kernels

Notebooks & interactive programming

Brief history: IPython→ Jupyter

- **2001:**
  IPython shell created by Fernando Pérez as an enhanced interactive Python environment
- **2011:**
  IPython Notebook introduced - first browser-based computational interface
- **2014:**
  The "Big Split" - Project Jupyter spins off; IPython becomes the Python kernel
- **2015–2017:**
  Growth of kernels (~100+ languages supported), community expansion
- **2018:**
  JupyterLab 1.0 released - modular, extensible, multi-document interface
- **2023–2025:**
  JupyterLab 4.x, real-world scalability, rich extension ecosystem



IPython console - where it all began

Notebooks & interactive programming

The JuPyteR Project

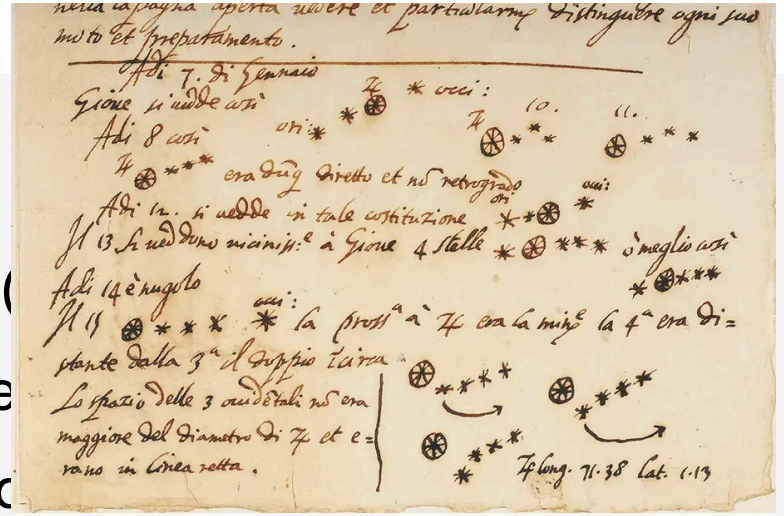
Julia + Python + R

Three core languages of scientific computing

Also References

🪐 Galileo's notebooks documenting Jupiter's moons

Symbolizing exploration and discovery



Key Components

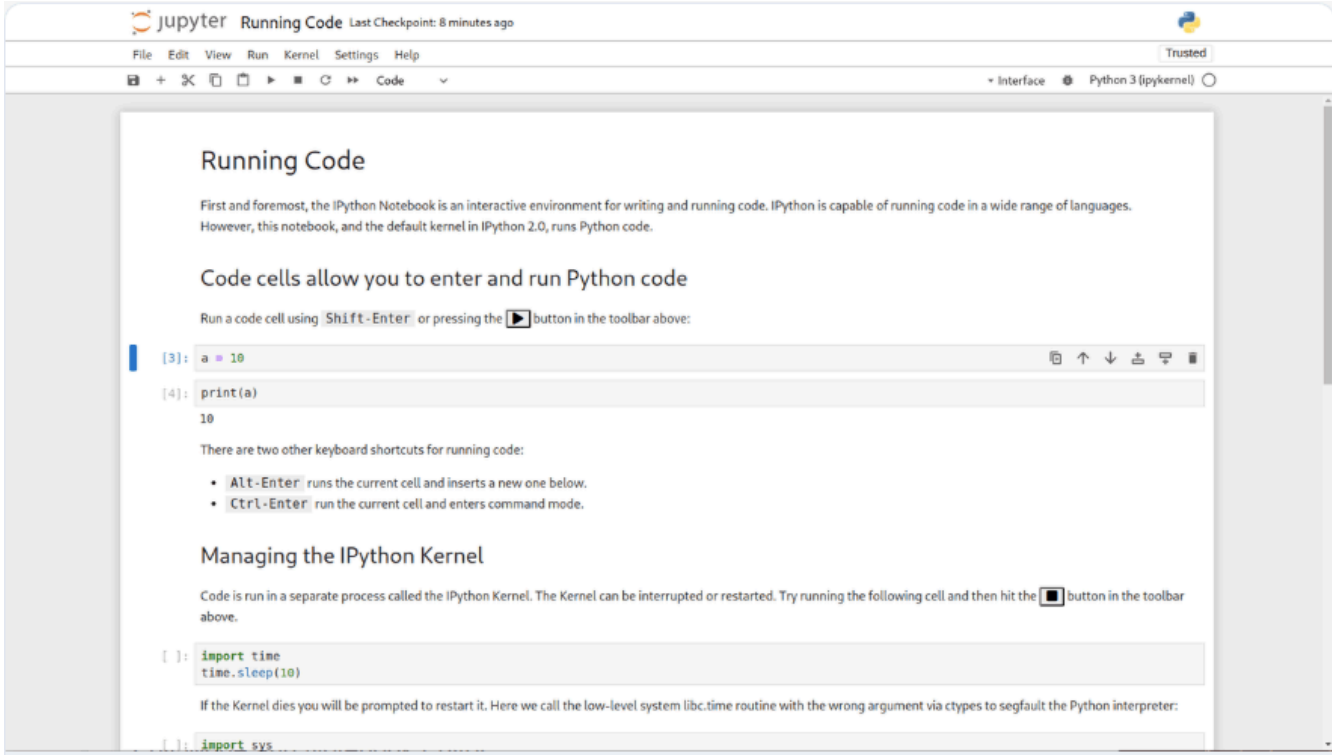
- **Jupyter Notebook:** Classic notebook interface (
- **JupyterLab:** Next-generation IDE-like interface
- **JupyterHub:** Multi-user server for teams and clo
- **Kernels:** Language execution backends (100+ languages supported)

Notebooks & interactive programming

Classic Jupyter Notebooks

Core concepts:

- **Cells**: Code cells (execute Python/R/Julia), Markdown cells (text, math), Raw cells (unformatted text);
- **Kernel**: Compute engine that executes code;
- **Toolbar**: Run cell, stop, restart kernel, change cell type, save...
- **Outputs**: Text, tables, plots (matplotlib, seaborn), interactive widgets



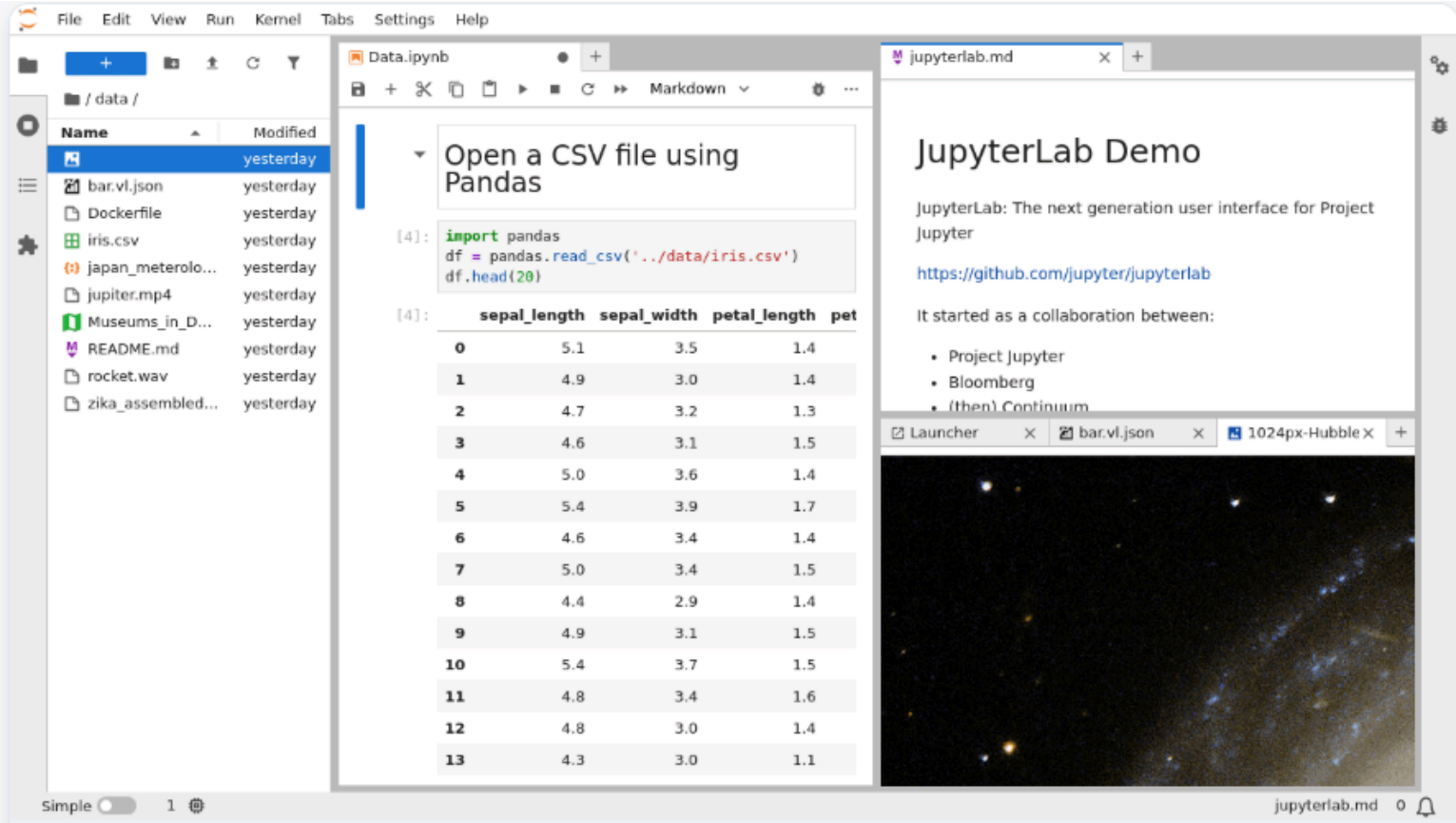
Jupyter Notebook interface with toolbar, menu, and cell structure

Run on a browser or integrated in IDE

Notebooks & interactive programming

Jupyter Lab

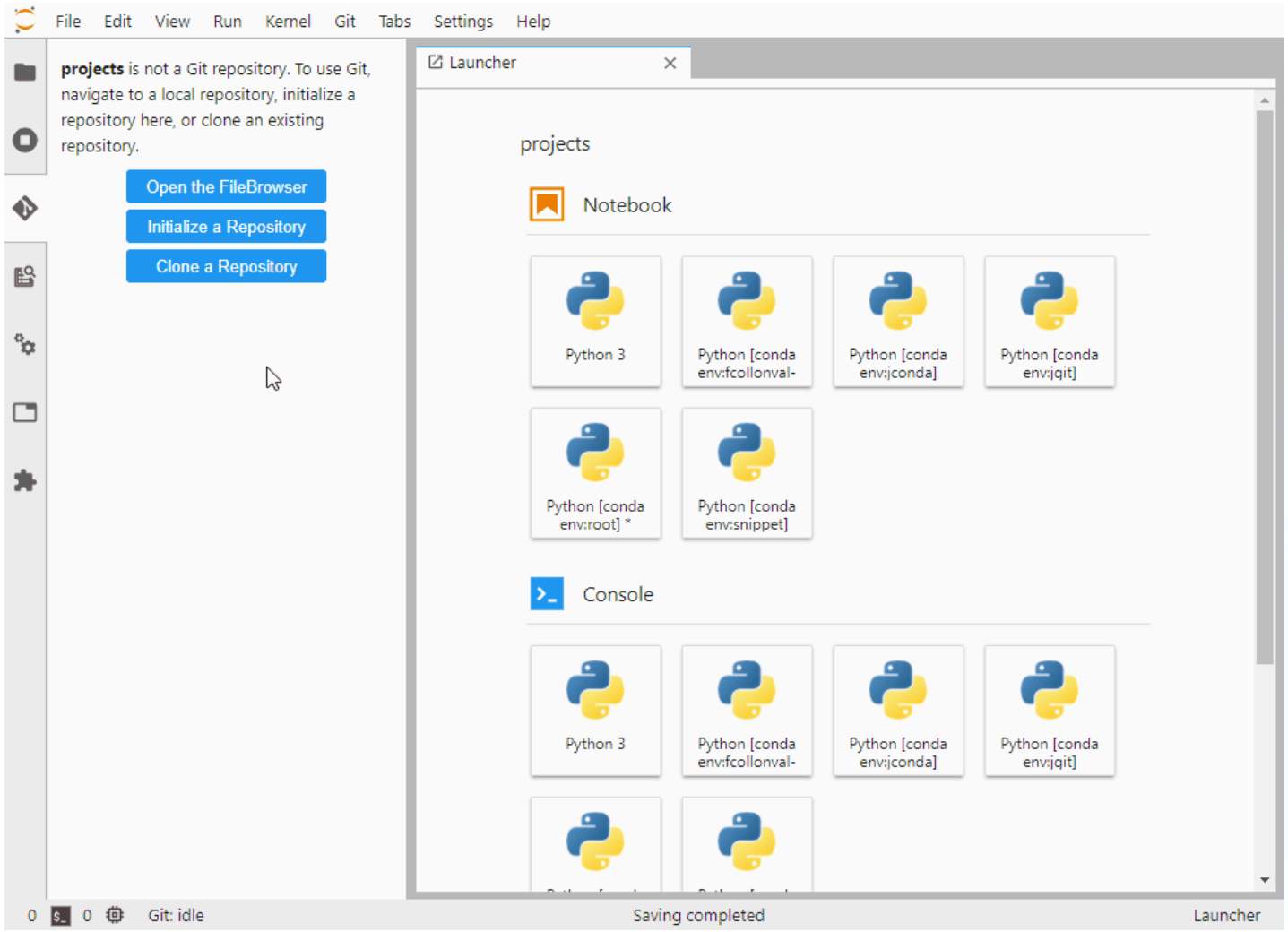
- **Multi-document layout:** Tabs, side-by-side notebooks, terminals, and text editor in a single interface
- **File browser:** Navigate files and directories without leaving your workspace
- **Command palette:** Quick access to all commands with Cmd/Ctrl+Shift+P
- **Split views:** Compare datasets or code in multiple panes
- **Integrated consoles:** Python console and system terminal available alongside notebooks
- **Extensible:** Powerful plugin architecture supports themes, extensions, and custom workflows



JupyterLab interface with file browser, notebook, and terminal

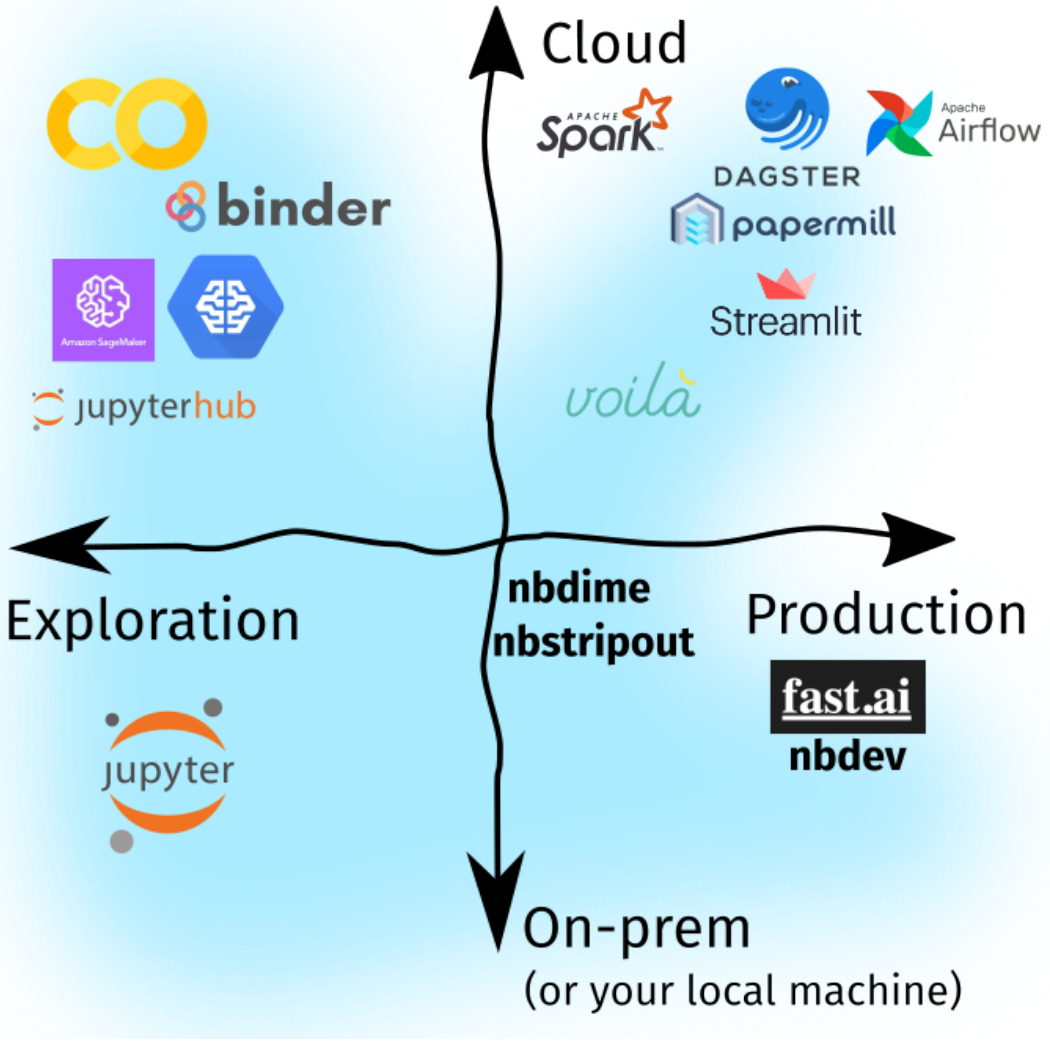
Run on a browser

Notebooks & interactive programming



With extensions, JLab becomes a complete IDE

Extensions & Ecosystem



Notebooks & interactive programming

Workflow: notebooks & scripts

Jupyter (Exploratory)

1. Load data interactively
2. Explore & visualize with immediate feedback
3. Clean & transform data incrementally
4. Iterate on hypotheses with cell-level execution
5. Narrate results with markdown & visualizations

Strengths:

- ✓ Interactivity with immediate feedback
- ✓ Rich visualization capabilities
- ✓ Narrative documentation with code
- ✓ Rapid prototyping & experimentation
- ✓ Excellent for data discovery & teaching

Python Script (Production)

1. Plan modules & architecture
2. Write functions, classes & CLI interfaces
3. Implement tests, error handling & logging
4. Execute in batch mode or scheduled runs
5. Package & deploy as reusable components

Strengths:

- ✓ Automation & reproducibility
- ✓ Scalability for large datasets
- ✓ Better version control integration
- ✓ Modular, reusable code structure
- ✓ Optimized for production pipelines

Notebooks & interactive programming

Workflow: notebooks & scripts

Feature	Jupyter Notebook (.ipynb)	Python Script (.py)
Execution	Cell-by-cell, non-linear	Top-to-bottom, sequential
State	Variables persist in memory	Fresh start each run
Output	Inline, visual, interactive	Terminal/console only
Documentation	Rich text, equations, images	Comments only (#)
Best For	Exploration, analysis, teaching	Production, automation, deployment

Best use for Notebooks:

- ✓ Exploratory data analysis
- ✓ Interactive teaching and learning
- ✓ Data visualization and reporting
- ✓ Communicating research findings

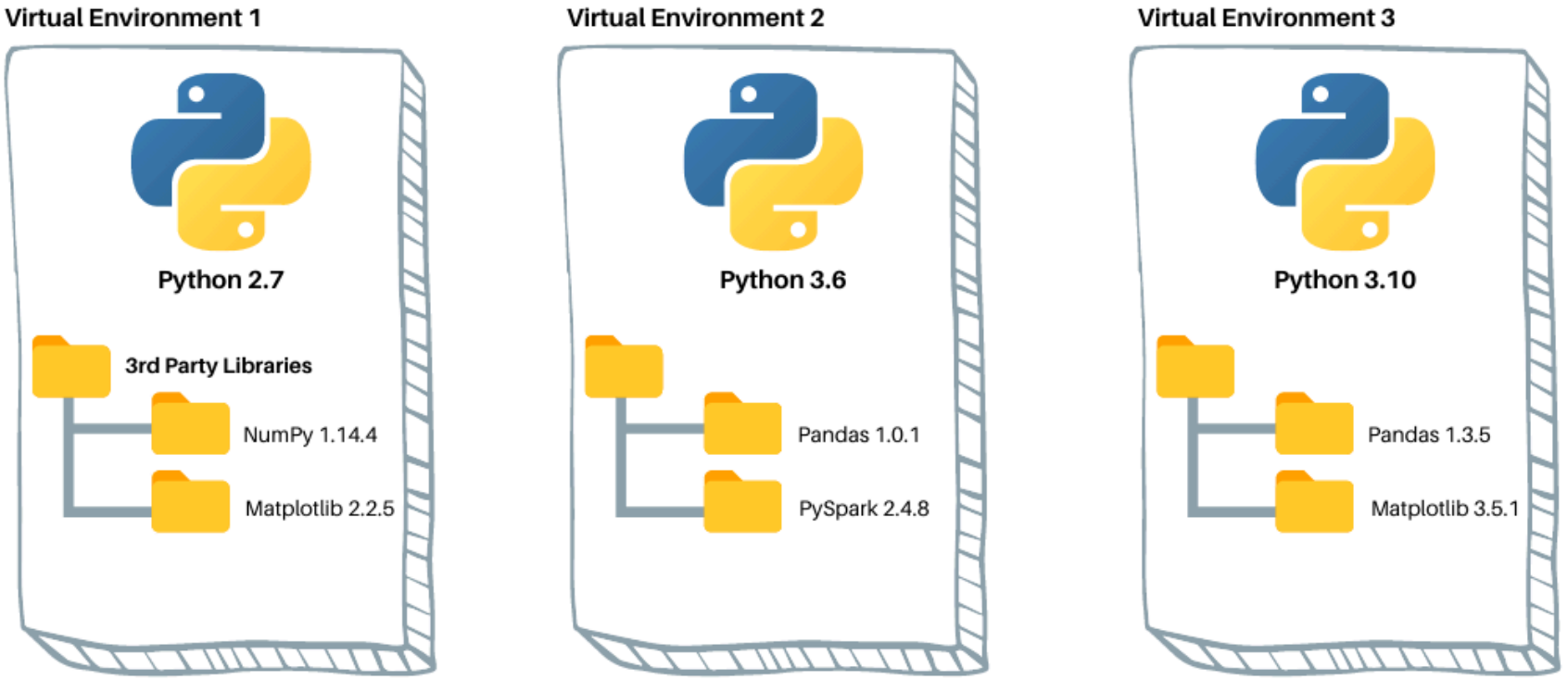
Best use for Scripts:

- ✓ Production data pipelines
- ✓ Automated batch processing
- ✓ Reproducible workflow engines
- ✓ Software library development

Virtual Environments

What and why?

- An **isolated Python workspace** on your computer
- Contains its own Python interpreter and packages
- Separate from your system Python installation



Key Benefits

- **Avoid C**
- **Reprod**
- **Clean S**
- **Safe Ex**

s

g projects

Virtual Environments

Creating with **venv**

Built-in Python Tool (Python 3.3+)

Step 1: Create the environment

```
python -m venv myproject_env
```

Step 2: Activate it

```
# Windows
myproject_env\Scripts\activate

# Mac/Linux
source myproject_env/bin/activate
```

✅ You'll see `(myproject_env)` in your terminal prompt

Virtual Environments

Daily workflow

Common Operations:

Install packages:

```
pip install numpy scipy pandas matplotlib
```

Check installed packages:

```
pip list
```

Deactivate when done:

```
deactivate
```


💡 **Tip:** Always activate your environment before working on a project!

Virtual Environments

Make Your Analysis Reproducible

Export your dependencies:

```
pip freeze > requirements.txt
```

Share this file with collaborators! They can recreate your environment:

```
pip install -r requirements.txt
```

> 🔬 **Research Best Practice:** Always include `requirements.txt` with your published code/data!

Virtual Environments



Conda environments

Popular in Scientific Computing (via Anaconda/Miniconda)

Create:

```
conda create -n myproject python=3.10
```

Activate:

```
conda activate myproject
```

Install packages:

```
conda install numpy scipy matplotlib bioconda::biopython
```

✨ **Key Advantage:** Can manage non-Python dependencies (R packages, system libraries, bioinformatics tools)

Virtual Environments

Poetry

Modern dependency management with **poetry**

All-in-One Dependency & Environment Manager

Initialize a project:

```
poetry new my-research-project
cd my-research-project
```

Add dependencies:

```
poetry add pandas biopython matplotlib
```

Activate environment:

```
poetry shell
```

✨ **Features:** Automatic virtual env creation, dependency resolution, lock files for reproducibility

pyproject.toml

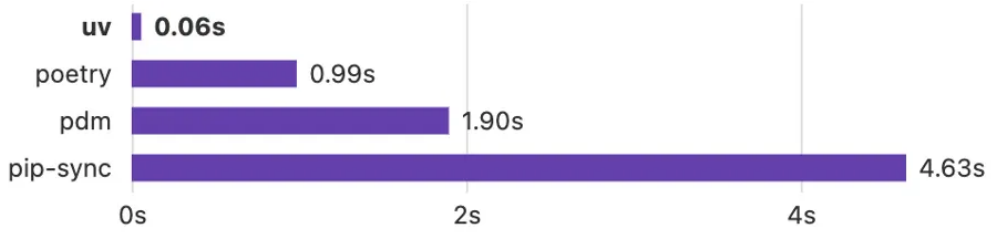
```toml
[tool.poetry]
name = "ml-demo"
version = "0.1.0"
description = ""
authors = ["Your Name <you@example.com>"]
readme = "README.md"
package-mode = false

[tool.poetry.dependencies]
python = "^3.12"
numpy = "^2.0.1"
pandas = "^2.2.2"
scikit-learn = "^1.5.1"
fastapi = "^0.111.1"
gunicorn = "^22.0.0"


[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

Virtual Environments

uv (New & Fast!)



Ultra-Fast Pip & Environment Replacement (by Astral)

Create environment:

```
uv venv myproject_env
```

Activate (same as venv):

```
source myproject_env/bin/activate  # Mac/Linux
myproject_env\Scripts\activate      # Windows
```

Install packages (10-100x faster!):

```
uv pip install pandas numpy matplotlib
```

🚀 **Speed:** Written in Rust, dramatically faster than pip!

Virtual Environments

Additional Tools Worth Knowing:

More options

🐍 Pipenv

Combines pip + virtualenv

```
pipenv install pandas
pipenv shell
```

Good for: Simple projects, easy lockfiles

📦 virtualenvwrapper

Extensions for managing multiple venvs

```
mkvirtualenv myproject
workon myproject
```

Good for: Managing many environments easily

🎯 pyenv

Manage multiple Python versions

```
pyenv install 3.11.0
pyenv virtualenv 3.11.0 myenv
```

Good for: Testing across Python versions

🔒 Rye

Comprehensive Python project manager

```
rye init myproject
rye add pandas
```

Good for: All-in-one project management

Virtual Environments

Quick comparison

Feature	Conda	Venv	UV	Poetry
Environment Mgmt	✅	✅	✅	✅
Package Mgmt	✅ (Conda)	❌ (uses pip)	✅ (pip-compatible)	✅ (Poetry)
Non-Python Deps	✅	❌	❌	❌
Lockfiles	❌	❌	✅ (`uv pip compile`)	✅ (`poetry.lock`)
Speed	⏳ Slow	⚡ Fast (envs)	⚡⚡⚡ Blazing fast	⏳ Moderate
CI/CD Optimized	❌	⚠️ (requires pip)	✅	⚠️
Packaging/Publish	❌	❌	❌	✅
Built-in Python	No	Yes (3.3+)	No	No

https://medium.com/@shugaoye/a-comparison-of-python-environment-and-package-management-tools-c34cd13514c1

Virtual Environments

Quick comparison

Action	Conda	Venv + pip	UV	Poetry
Add Package	`conda install`	`pip install`	`uv pip install`	`poetry add`
Remove Package	`conda remove`	`pip uninstall`	`uv pip uninstall`	`poetry remove`
Lockfile	❌ (`environment.yml` only)	❌ (manual updates)	✅ `uv pip compile`	✅ Auto `poetry.lock`
Dev Dependencies	`conda install pytest`	`pip install pytest`	`uv pip install pytest`	`poetry add pytest --group dev`
Version Pinning	`conda install requests=2.32`	`pip install requests==2.32`	`uv pip install requests==2.32`	`poetry add requests@2.32`
Config File	`environment.yml` (manual export)	`requirements.txt` (manual)	`pyproject.toml` + `requirements.lock`	Auto `pyproject.toml` / `poetry.lock`

https://medium.com/@shugaoye/a-comparison-of-python-environment-and-package-management-tools-c34cd13514c1

Virtual Environments

Quick comparison

Tool	Speed	Setup	Best For	Learning Curve
venv	⚡⚡⚡ Fast	✅ Built-in	Beginners, simple projects	⭐ Easy
Conda	🐌 Slower	❌ Needs install	Bioinformatics, data science	⭐⭐ Medium

https://medium.com/@shugaoye/a-comparison-of-python-environment-and-package-management-tools-c34cd13514c1

Virtual Environments

Best practices

✅ Do's:

- **One environment per project**
- **Always activate before working**
- **Keep requirements.txt/pyproject.toml updated**
- **Name environments descriptively** (e.g., `rna_seq_2024`)
- **Add env folders to .gitignore**
- **Add data, figures and big files to .gitignore**
- **Document which tool you use** in your README

❌ Don'ts:

- **Installing without activating** → Goes to system Python!
- **Committing env folders and data to Git** → Huge repo size
- **Using global Python for everything** → Dependency chaos
- **Mixing tools** (e.g., conda + pip carelessly) → Conflicts

⚠️ Common Mistake:

Installing packages without activating the environment first – they'll go to your system Python!

🧬 **Remember:** Virtual environments = organized, reproducible, professional research code!

Virtual Environments

Summary & Key Takeaways

1. **Why Virtual Environments?**
   - Isolate projects, avoid conflicts, ensure reproducibility - essential for good research practices!

2. **Many Tools Available**
   - **venv** (simple), **Conda** (scientific), **Poetry** (modern), **uv** (fast) - choose what fits your needs!

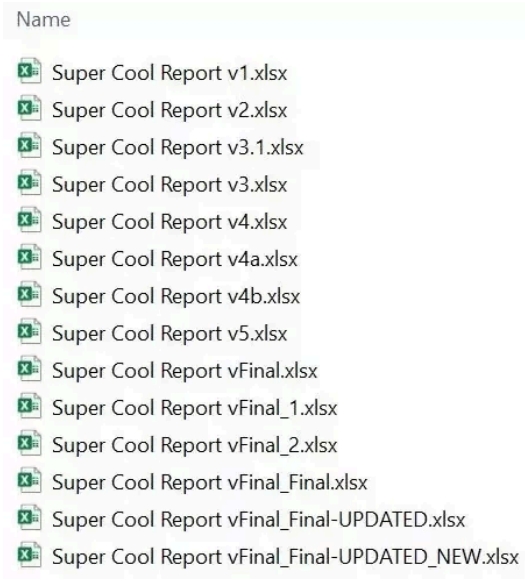
3. **Document Dependencies**
   - Use `requirements.txt`, `environment.yml`, or `pyproject.toml` to share your exact setup

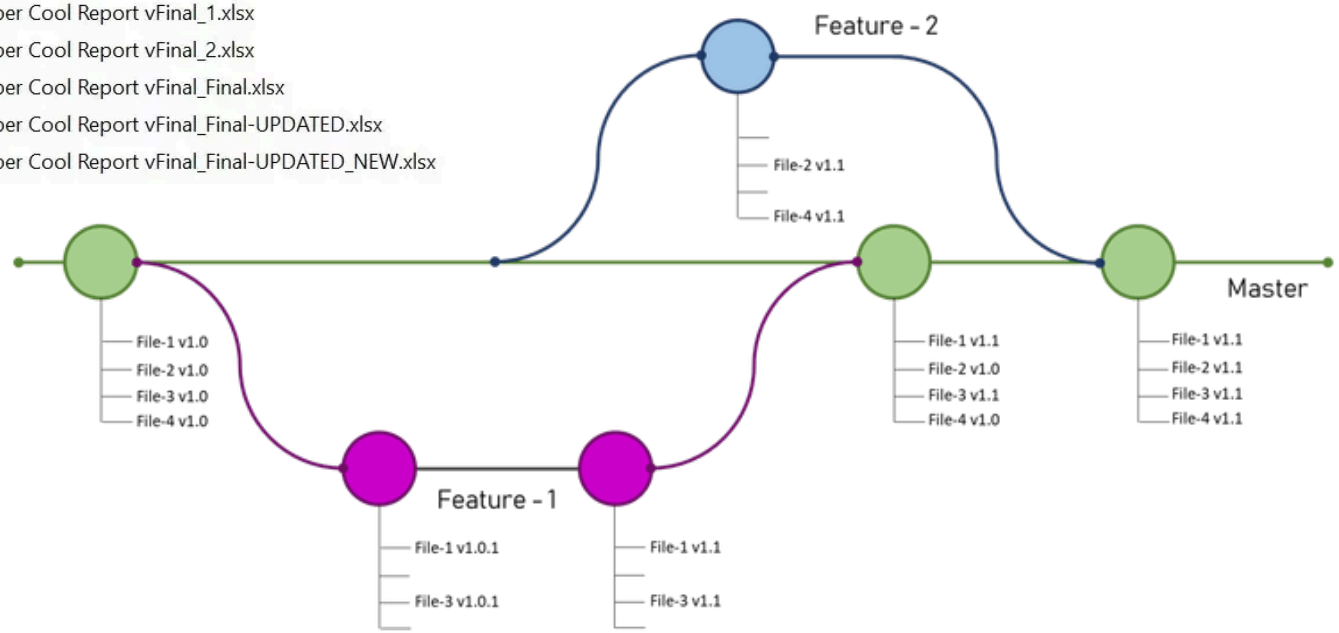
4. **Make It a Habit**
   - Create a new environment for every project from day one - **future you will be grateful!**

🧬 **Remember:** Virtual environments = organized, reproducible, professional research code!

Git & Version Control

Essential Tools for Scientific Programming







A beginner's guide to **tracking**, **managing**, and **collaborating** on code

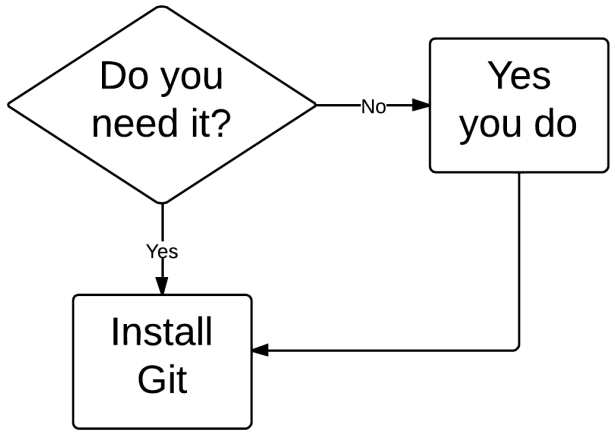
Git & Version Control

What is Version Control?

A **system that tracks changes to files over time**

- 📝 Records **who** made changes, **what** changed, and **when**
- ⏮️ Allows reverting to previous versions at any time
- 🤝 Enables collaboration without file conflicts

Version Control Flowchart



1. **Reproducibility:** Track which code produced which results
2. **Transparency**: Document all changes to methods and analysis
3. **Backup:** Never lose work, recover any version
4. **Collaboration:** Work with colleagues and teams without file conflicts

Git & Version Control

1980s-1990s: CVS

Centralized, single server

2000s: SVN/Subversion

Improved CVS, still centralized

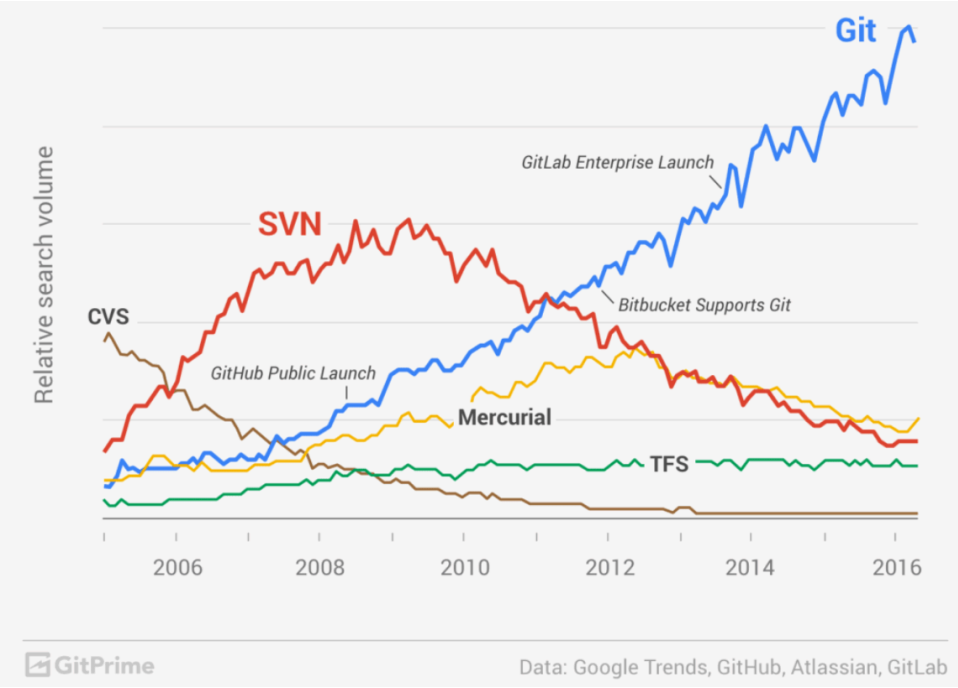
2005: Git

Distributed, fast, powerful — became standard

2008: GitHub

Made Git accessible with web interface

Evolution of Version Control Systems



Git & Version Control

The Modern Era (2008-Present)

2008: GitHub Launch

Made Git accessible via web interface, revolutionized collaboration

2011: GitLab Founded

Open-source alternative with self-hosting option, DevOps focus

2013: Bitbucket Adds Git Support

Atlassian's platform embraces Git, integrates with Jira

2018: Microsoft Acquires GitHub ($7.5B)

Major shift: Microsoft embraces open source, deep VS Code integration

2021-2023: AI Integration Era

GitHub Copilot, GitLab Duo, AI-assisted code review and generation

2024-Present: Dominant Era

Git is universal standard • GitHub: 100M+ users • GitLab, Bitbucket thrive in enterprise

Key Takeaway: *Git won the version control wars. Today's competition is between hosting platforms (GitHub, GitLab, Bitbucket), not version control systems.*

Git & Version Control

What is Git?

A **distributed version control system** for tracking changes in source code

- 📅 Created by **Linus Torvalds** in 2005 (Linux kernel development)
- 🆓 **Free and open source**
- 💻 **Works locally** on your computer
- 📁 **Tracks any file type** (code, documents, data)
- 🏆 **Industry standard** — used by millions of developers

⚠️ Important Distinction

Git ≠ GitHub

Git is the tool (software on your computer) • GitHub is a hosting platform (website for sharing repositories)

Git & Version Control

📂 **Repository (Repo):** The project folder that Git tracks — contains all files and complete history

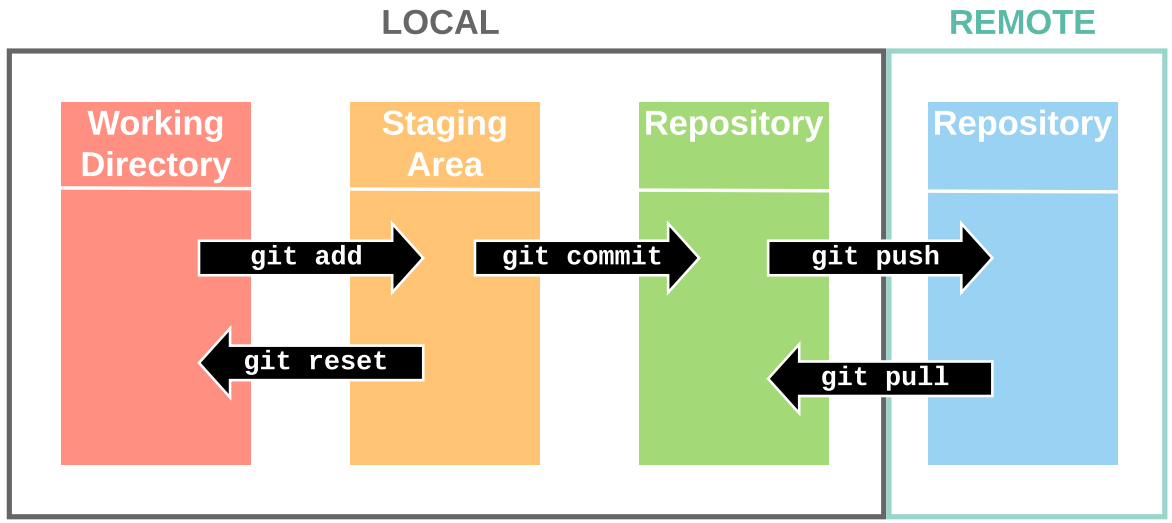
☁️ **Remote:** A version of the repository hosted online (e.g., GitHub, GitLab) — enables collaboration and backup

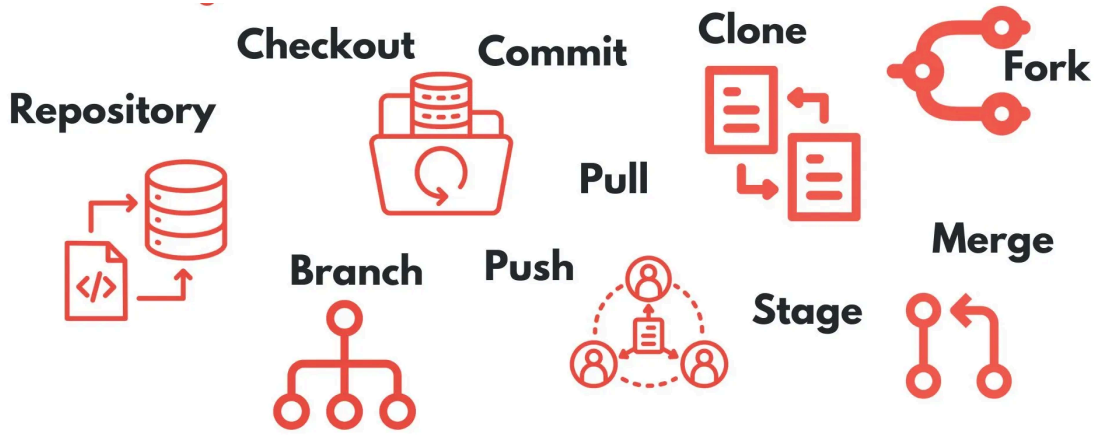
📸 **Commit:** A snapshot of changes with a descriptive message — Git's save point

🌳 **Branch:** An independent line of development — test ideas without affecting main code

🔀 **Merge:** Combining changes from different branches into one — integrate new work

Core Git Concepts





Git & Version Control

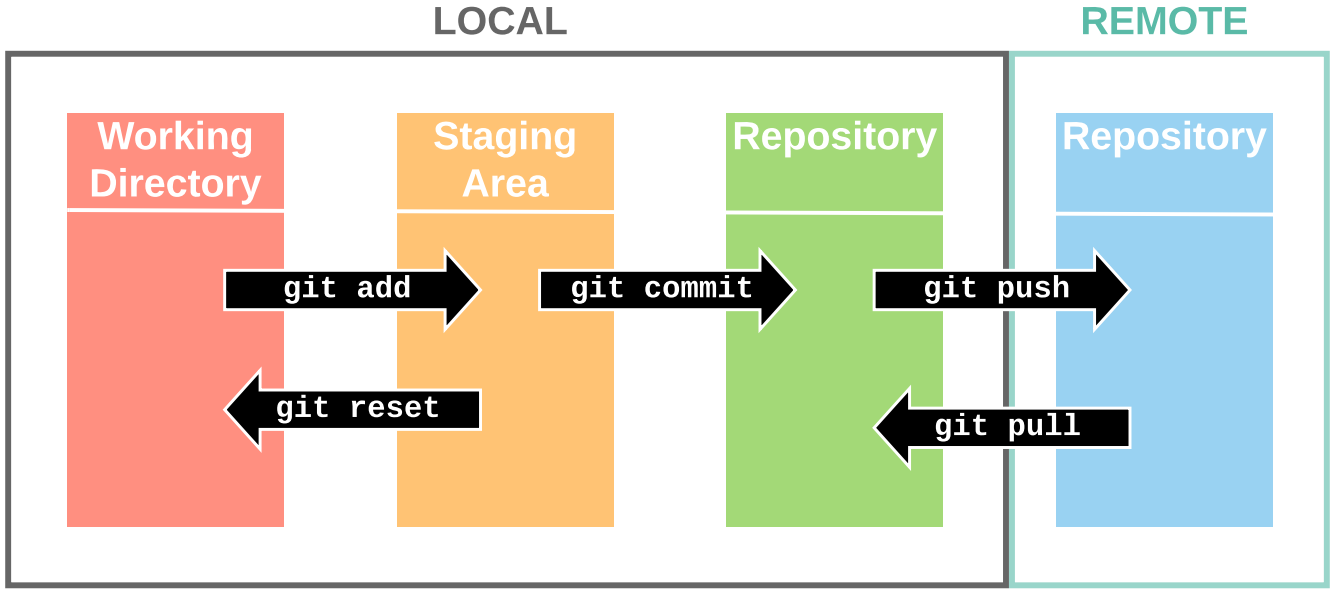
```
git init
Initialize a new Git repository
```

```
git clone [URL]
Initialize a new Git repository cloning the remote
```

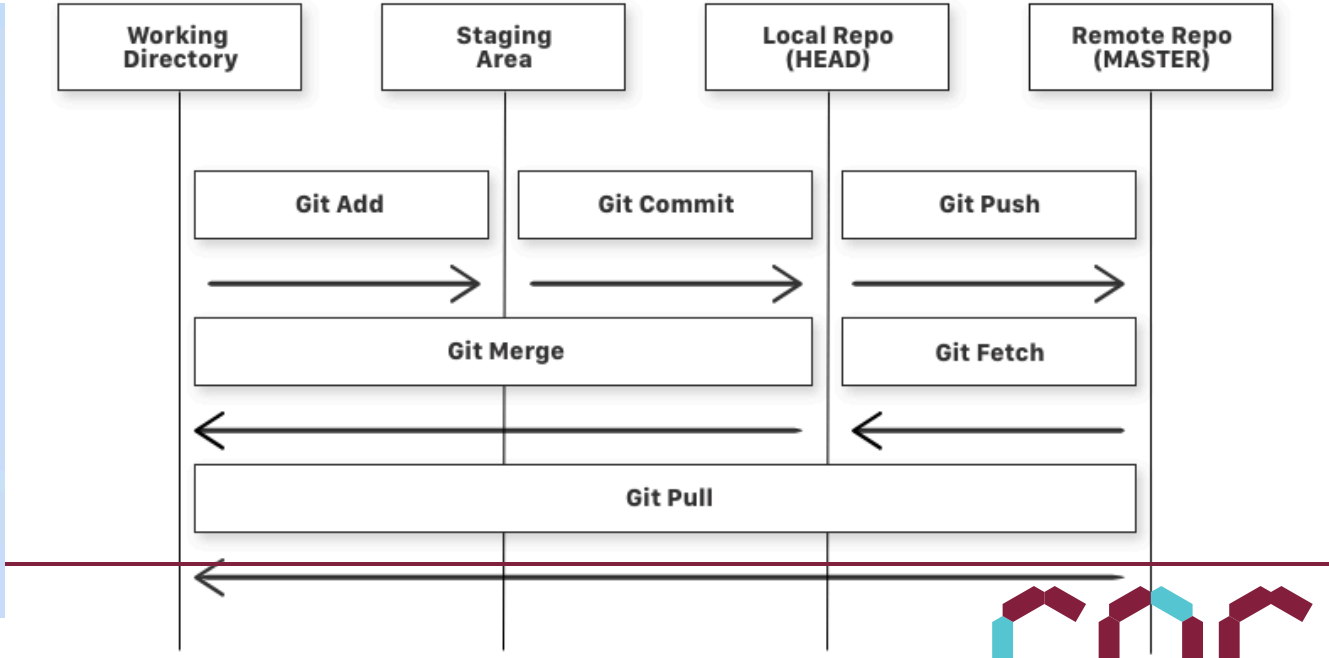
```
git remote add origin [URL]
Connect to remote repository (if not cloned)
```

```
git status
Check file status and changes
```

Essential Git Commands





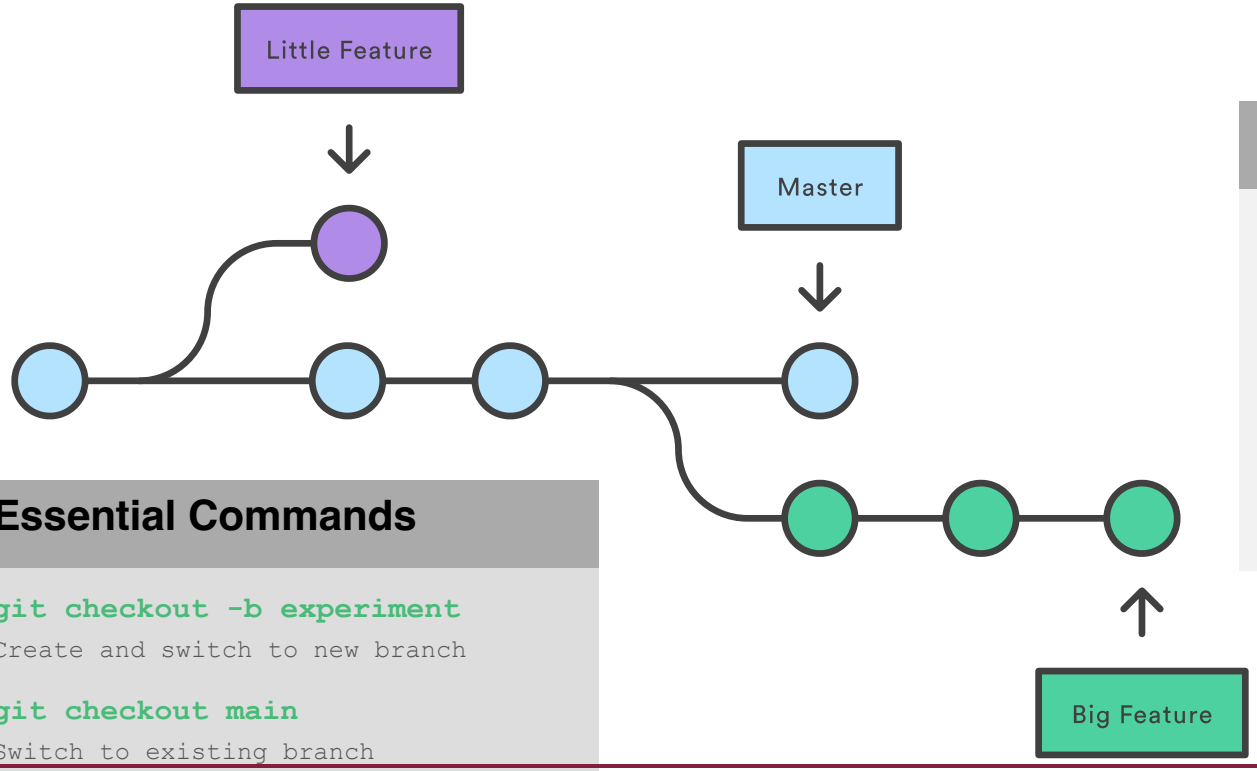


https://levelup.gitconnected.com/git-for-beginners-basic-terms-essential-commands-how-to-work-platforms-and-git-vs-github-755b55760e62

Git & Version Control

Why Branch?

- Test new analysis methods without breaking working
- Work on multiple features simultaneously
- Collaborate safely — each person works on their own
- Experiment with different statistical approaches



Example: Test

1. Create bra
2. Make char
3. Test thoro
4. If it works:
5. If it doesn'

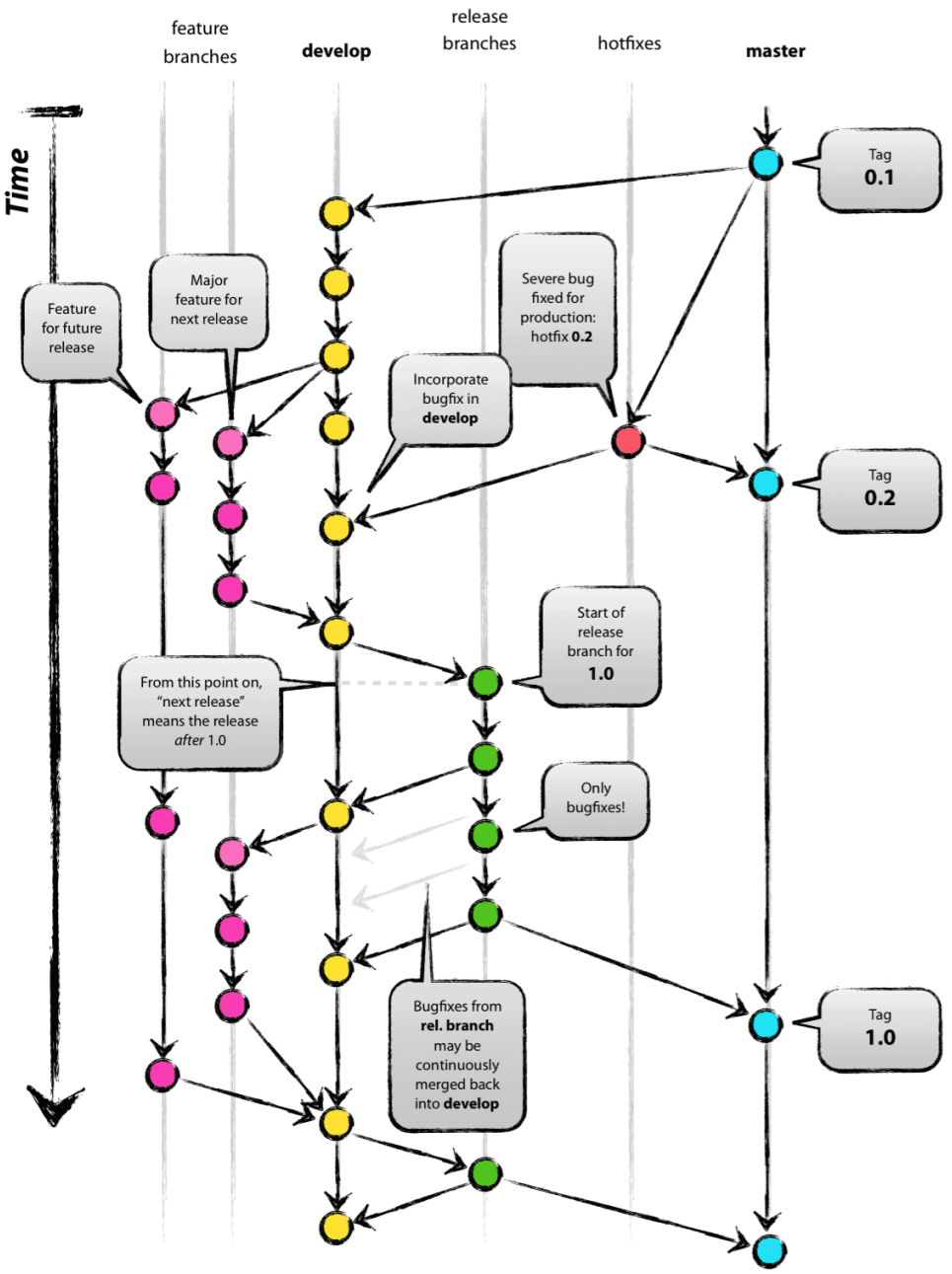
Essential Commands

```
git checkout -b experiment
```
Create and switch to new branch

```
git checkout main
```
Switch to existing branch

```
git merge experiment
```
Merge branch into current

Branching & Merging



Git & Version Control

Git Tools for Windows Users

Command Line (Git Bash)



✓ Pros:

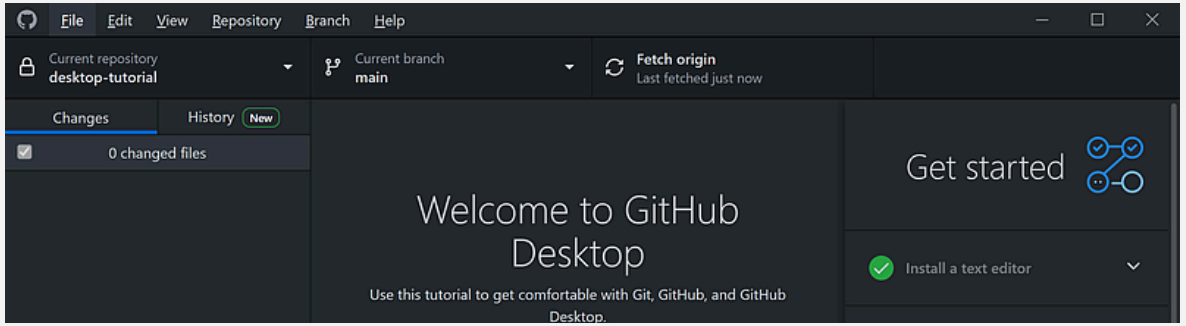
- Full functionality — access all Git features
- Powerful and flexible
- Works everywhere (transferable skills)
- Industry standard

✗ Cons:

- Learning curve for beginners
- Must memorize commands
- Text-based interface

Even simpler: use IDE integration

Graphical Interface (GitHub Desktop)



✓ Pros:

- User-friendly for beginners
- Visual diffs — see changes side-by-side
- No commands to memorize
- **Great starting point for scientists**

✗ Cons:

- Limited to common operations
- Less control than command line

🌟 Recommended for this course!

Git & Version Control

Git for Scientific Workflows

📊 Track Analysis Code

Document every data processing step — know exactly which code version produced which results

🔬 Reproducible Research

Link code versions to publications — reviewers and readers can reproduce your exact analysis

👥 Collaborate Effectively

Multiple researchers on same project — work simultaneously without conflicts or overwriting files

🧪 Experiment Safely

Try different statistical approaches using branches — test without fear of breaking working code

📣 Share Openly

Publish code on GitHub for transparency — enhance credibility and enable others to build on your work

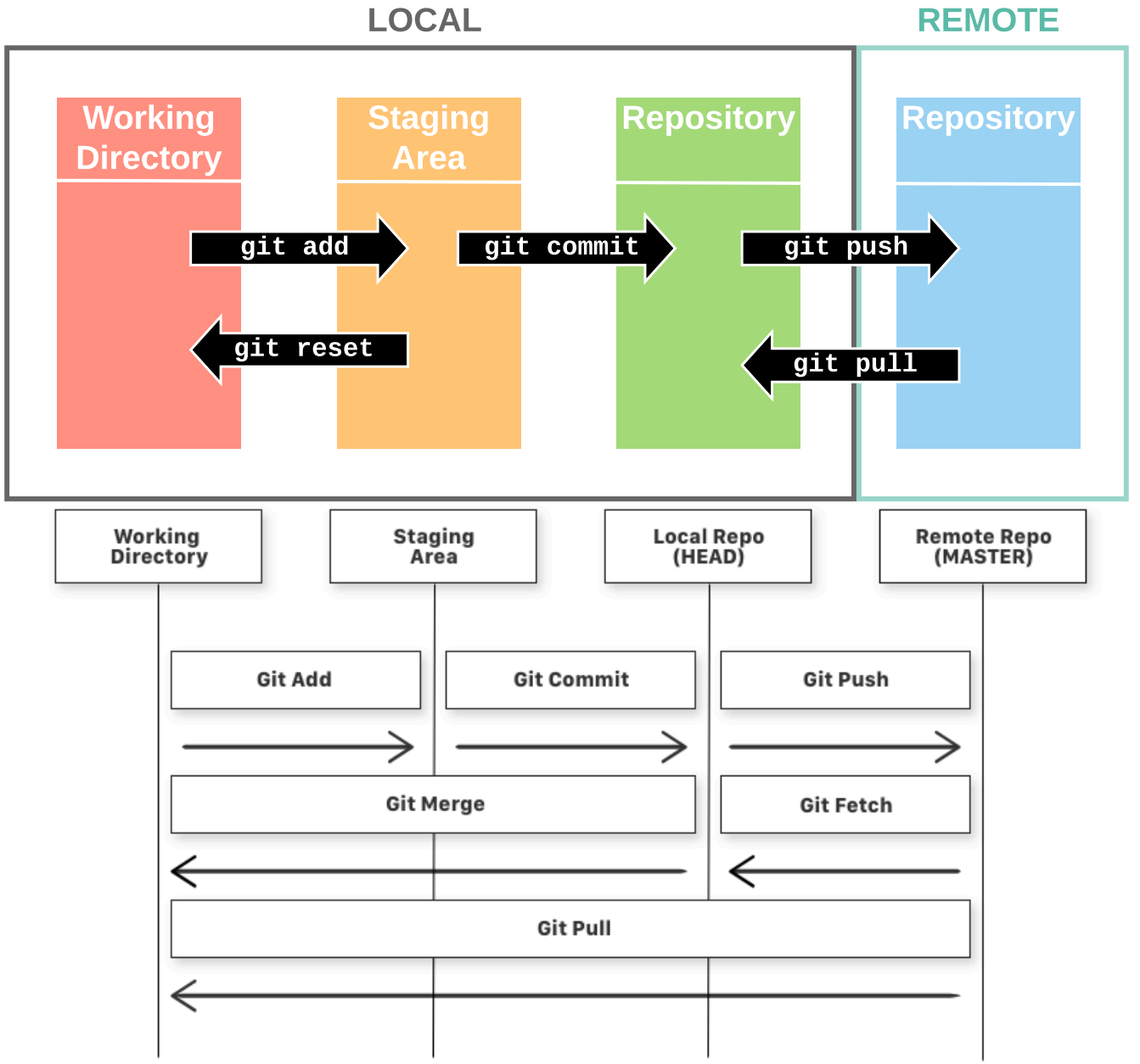
✨ Best Practices for Scientists

- **Commit often** — small, logical changes
- **Write clear messages** — "Fixed t-test calculation" not "updates"
- **Push regularly** — backup your work to remote
- **Use meaningful branch names** — "new-stats-method" not "test123"

Git & Version Control

1. Start working, get the latest code version
   - `git fetch or git pull`
2. Complete relevant modifications and logical additions
   1. **Stage**:
      - `git add .`
   2. **Commit to local**:
      - `git commit -m "msg"`
   3. **Push to remote:**
      - `git push origin [branch]`

The Git Daily Workflow



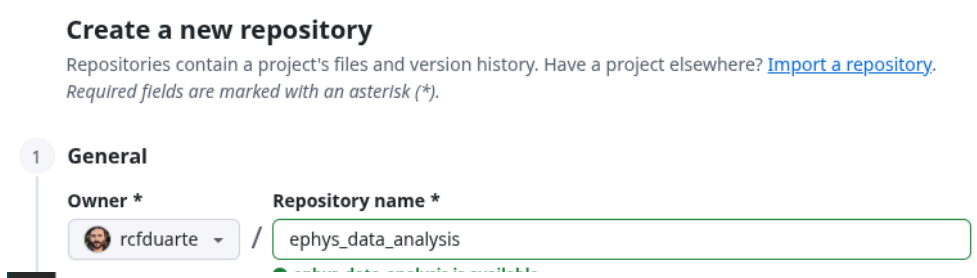
Computational Research Workflow

New Project: ephys_data_analysis

(data analysis, modeling, simulation, etc)

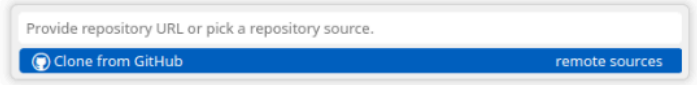
→ 1. Create a Git Repo

https://github.com/rcfduarte/ephys_data_analysis.git



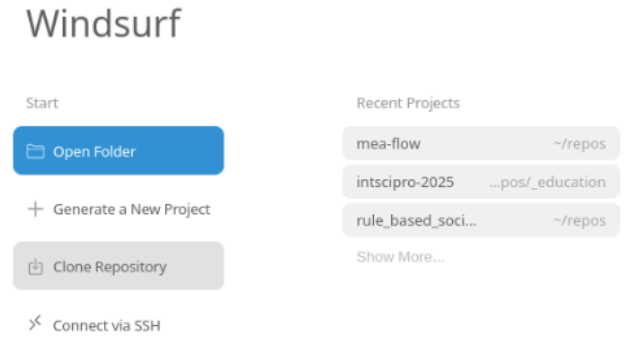
→ 2. Clone Repo in local folder

```
cd /home/projects/
git clone https://github.com/rcfduarte/ephys_data_analysis.git
```



→ 3. Create project-specific venv

```
conda create -n ephys_data_analysis python=3.12
conda activate ephys_data_analysis
```



→ 4. Install dependencies as required

```
conda install numpy scipy matplotlib pandas
```

Computational Research Workflow

New Project: ephys_data_analysis

(data analysis, modeling, simulation, etc)

→ 5. Document dependencies and add to Rep

```
conda env export > environment.yml
OR
conda list --explicit > requirements.txt
```

→ 6. Setup basic project file/folder structure

→ 7. Exclude heavy folders from git tracking

Careful with storing large data files in Repo

- Ideally keep data and code separate
- if not, keep data folder in .gitignore

→ 8. Commit and push

```
git add .
git commit -m "First commit; complete folder structure; complete gitignore; ready to role"
git push origin main
```

```
# Python
__pycache__/
*.so
env/
venv/
ENV/
.venv

# Jupyter
.ipynb_checkpoints
*.ipynb_checkpoints/

# Data files (if large)
*.csv
*.xlsx
*.h5
*.hdf5
data/

# Results
results/
output/
figures/

# IDEs
.vscode/
.idea/
*.swp
*.swo
*~

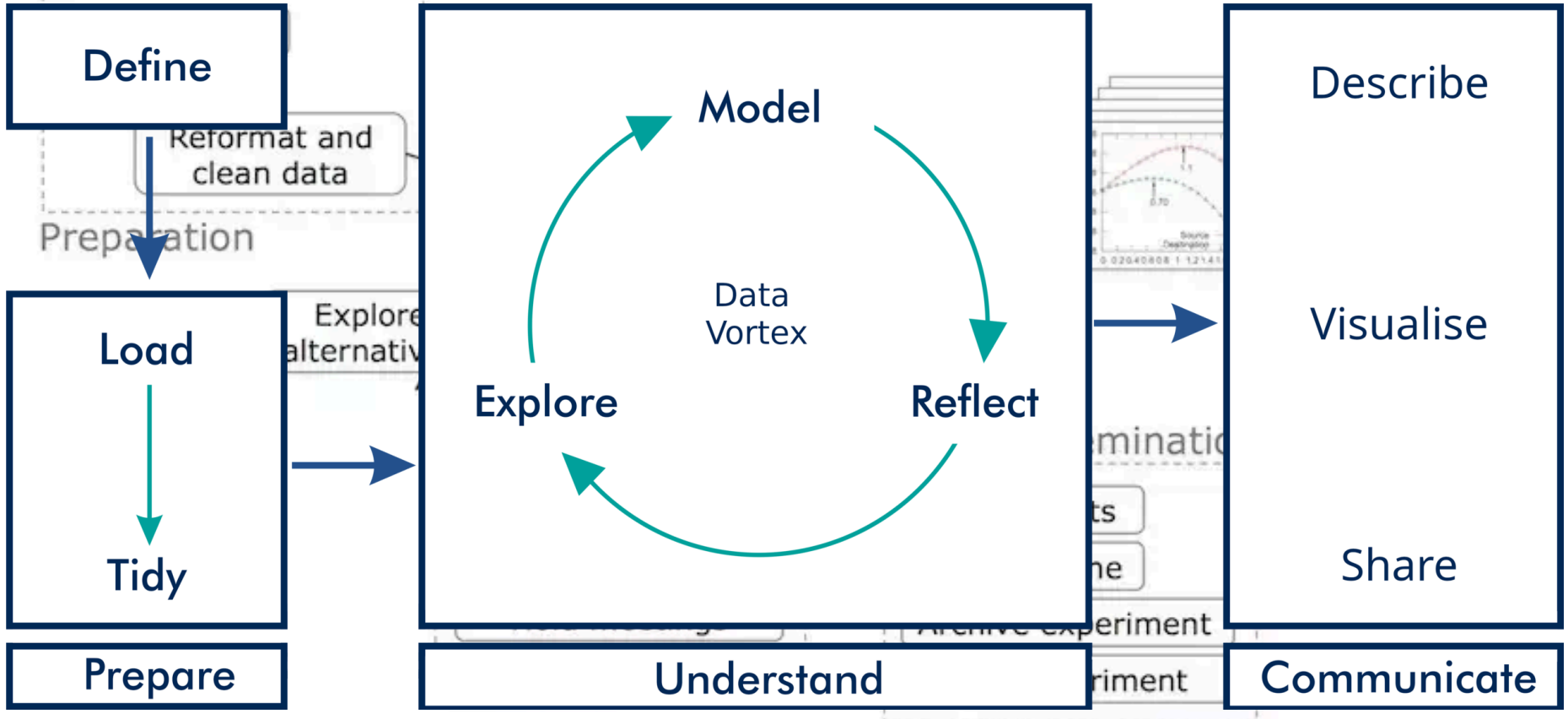
# OS
.DS_Store
Thumbs.db
```

Computational Research Workflow

New Project: ephys_data_analysis

(data analysis, modeling, simulation, etc)

Commit and push regularly (at least after every "session") and update dependencies if needed



Have Fun!!

Thank you!





This week's materials were prepared with the help of Claude Sonnet 4.5, via GenSpark